

Progetto C2: Shell

Relazione tecnica sull'implementazione delle system call

per la gestione di file e processi in OS/161

Rebecca Di Quinzio (s360413), Tobia Franceschetti (s360414)

Indice

1. Obiettivo del progetto	3
2. Architettura generale	6
2.1 Sincronizzazione e concorrenza	6
3. Gestione dei file aperti: file descriptor table e openfile	7
3.1 Requisito della traccia e scelta progettuale	7
3.2 File descriptor table per processo	7
3.3 Struttura openfile	7
3.4 Inizializzazione di stdin, stdout e stderr	9
4. Implementazione delle system call sui file	10
4.1 open()	10
4.2 read() e write()	11
4.3 close()	13
4.4 lseek()	14
4.5 dup2()	15
4.6 chdir() e __getcwd()	16
5. Modello dei processi e gestione dei PID	17
5.1 Estensione di struct proc	17
5.2 processTable globale e assegnazione dei PID	18
5.3 getpid()	18
6. fork()	19
7. execv()	21
7.1 Differenza rispetto a runprogram()	22
8. waitpid(), _exit() e ciclo di vita dei processi	23
8.1 _exit() e stato zombie	23
8.2 waitpid() e sincronizzazione	23
8.3 Gestione degli orfani	25
8.4 Integrazione con il menu kernel	25
9. Gestione delle eccezioni fatali	26
10. Error handling e atomicità	27
10.1 Validazione dei puntatori utente	27
10.2 Errori restituiti	27
11. Verifica funzionale	28
12. Conclusioni e principali difficoltà riscontrate	28

1. Obiettivo del progetto

Il progetto C2 ha l'obiettivo di estendere OS/161 per supportare l'esecuzione contemporanea di più programmi utente, caricati dal file system e gestiti attraverso la shell. A tale scopo vengono implementate le system call per la gestione dei file e dei processi e la gestione delle eccezioni in user mode, garantendo il corretto trattamento degli errori senza provocare il crash del kernel.

PIATTAFORMA E CONFIGURAZIONE

- **Sistema operativo didattico:** OS/161
- **Architettura:** MIPS32
- **Simulatore:** System/161
- **File system utilizzato:** EMUFS, coerentemente con la possibilità prevista dalla traccia di limitare il progetto a questo file system.
- **Configurazione del kernel: PDSC2**, costruita a partire dalle configurazioni utilizzate nei laboratori precedenti svolti in aula, mantenendone le relative opzioni per comodità e coerenza con l'ambiente adottato durante il corso:
 - options dumbvm
 - options syscalls
 - options synch
 - options waitpid
 - options dumbvm_with_free
 - options pdsc2

STRUMENTI DI SVILUPPO

- **Visual Studio Code:** editor utilizzato per lo sviluppo e la modifica del codice sorgente.
- **GitHub:** repository remoto utilizzato per la gestione e la consegna del progetto.
- **Terminale Linux:** utilizzato per configurazione, compilazione ed esecuzione di OS/161 e dei relativi test.
- **System/161:** simulatore utilizzato per eseguire il kernel OS/161 su architettura MIPS.

FILE PRINCIPALI E REQUISITI COPERTI

File	Ruolo nel progetto
kern/syscall/file_syscalls.c	Implementazione delle syscall relative ai file: open, read, write, lseek, close, dup2, chdir, __getcwd, oltre alla gestione delle openfile.
kern/syscall/proc_syscalls.c	Implementazione delle syscall relative ai processi: getpid, fork, execv, waitpid e _exit.
kern/proc/proc.c	Gestione interna dei processi: creazione/distruzione, PID, relazione padre-figlio, terminazione, file descriptor table e gestione degli orfani.
kern/include/proc.h	Estensione della struct proc con lo stato necessario al progetto, tra cui PID, stato di terminazione, parent PID e file descriptor table.

kern/arch/mips/syscall/syscall.c	Dispatcher delle system call e gestione dell'interfaccia MIPS user/kernel, compreso il caso particolare di lseek().
kern/arch/mips/locore/trap.c	Gestione delle eccezioni fatali provenienti dallo user mode, tramite kill_curthread().
kern/main/menu.c	Integrazione con l'avvio/esecuzione dei programmi dal menu del kernel.
kern/conf/PDSC2	Configurazione utilizzata per compilare il kernel del progetto.

Requisito della traccia	Dove è implementato nel progetto	Descrizione
File I/O: open, read, write, lseek, close	kern/syscall/file_syscalls.c	Implementazione delle relative sys_*, utilizzando VFS/VOP e la struttura openfile.
Duplicazione dei descriptor: dup2	file_syscalls.c → sys_dup2()	newfd viene fatto puntare alla stessa openfile di oldfd, incrementandone il reference count e mantenendo quindi l'offset condiviso.
Directory corrente: chdir, getcwd	file_syscalls.c → sys_chdir() e sys___getcwd()	Gestione tramite VFS e directory corrente associata al processo.
stdin, stdout, stderr inizialmente su con:	proc_create_runprogram()	I fd 0, 1 e 2 vengono inizializzati rispettivamente in lettura, scrittura e scrittura verso la console.
PID: getpid + allocazione e recupero dei PID	proc.c, proc.h, proc_syscalls.c	processTable assegna e ricicla i PID; sys_getpid() restituisce quello di curproc.
Creazione dei processi: fork	proc_syscalls.c → sys_fork(); proc.c → proc_create_fork()	Crea una nuova struct proc, copia l'address space e il trapframe ed eredita cwd e file aperti.
Sostituzione del programma: execv	proc_syscalls.c → sys_execv()	Copia pathname/argv in kernel space, crea un nuovo address space, carica l'ELF, ricostruisce argv sul nuovo stack e passa al nuovo programma.
Attesa dei figli: waitpid	proc_syscalls.c → sys_waitpid(); proc.c → proc_wait()	Verifica la relazione padre-figlio, attende tramite semaforo, restituisce lo status e successivamente reclama il processo.
Terminazione: _exit	proc_syscalls.c → sys___exit(); proc.c → proc_finish_exit()	Memorizza lo status, rilascia address space e thread e gestisce zombie/orfani e risveglio del padre.
Eccezioni fatali user mode	kern/arch/mips/locore/trap.c → kill_curthread()	Termina soltanto il processo responsabile senza far cadere il kernel.
Dispatch e ABI delle syscall	kern/arch/mips/syscall/syscall.c	Riconosce il codice della syscall, ricostruisce gli argomenti secondo l'ABI MIPS e restituisce risultato/errore nei registri corretti.
Interfaccia kernel delle syscall	kern/include/syscall.h	Sono dichiarati i prototipi delle syscall implementate.
Concorrenza tra syscall	proc.c, file_syscalls.c	Utilizzo di spinlock, lock e semafori per proteggere process table, file table, openfile e sincronizzazione waitpid/_exit.

Gestione corretta degli errori	Tutte le sys_* + dispatcher	Controlli degli argomenti, propagazione di errori come EBADF, EINVAL, EFAULT, ENOMEM, ENPROC, ecc. e cleanup delle risorse.
Utilizzo della shell /bin/sh	Risultato dell'insieme di fork, execv, dup2, waitpid e file syscall	Consente alla shell di lanciare programmi, effettuare redirezioni e gestire processi anche in background.

2. Architettura generale

Il progetto è stato organizzato in due aree principali: gestione dei file aperti e gestione dei processi.

Per i file, ogni processo possiede una propria file descriptor table, contenuta nella struct proc. I file descriptor non puntano direttamente ai file del VFS, ma a strutture openfile, che mantengono lo stato dell'apertura, come vnode, offset e flag, e possono essere condivise tra più descriptor o processi quando necessario, ad esempio con dup2() e fork().

Per i processi, la struct proc è stata estesa con PID, relazione padre-figlio, stato di terminazione e informazioni necessarie a waitpid(). È stata inoltre introdotta una process table globale, utilizzata per associare ogni PID alla relativa struttura di processo e gestirne creazione, ricerca e rilascio.

Le strutture condivise sono protette con lock, spinlock e semafori specifici, in modo da sincronizzare solo le risorse effettivamente condivise senza bloccare globalmente tutte le system call.

2.1 Sincronizzazione e Concorrenza

L'implementazione tiene conto del fatto che più thread, appartenenti anche a processi diversi, possono eseguire contemporaneamente system call nel kernel. Per evitare race condition senza serializzare inutilmente operazioni indipendenti, non è stato utilizzato un unico lock globale: ogni struttura condivisa è protetta dal meccanismo di sincronizzazione più adatto.

I principali meccanismi utilizzati sono:

Meccanismo	Stato protetto / funzione	Primitiva	Motivazione
processTable.lk	Process table, associazione PID → proc, stato dei processi	Spinlock	Protegge operazioni molto brevi, come ricerca, assegnazione e rilascio dei PID, che non richiedono operazioni bloccanti.
p_filetable_lock	File descriptor table del singolo processo	Lock	Rende atomiche modifiche alla tabella durante operazioni come close(), dup2() e copia della tabella in fork().

of_lock	Stato della singola openfile, soprattutto offset e I/O	Lock	Più descriptor/processi possono condividere la stessa openfile; il lock impedisce aggiornamenti concorrenti dell'offset.
system_file_table_lock	Tabella globale delle openfile e reference count	Lock	Protegge allocazione e rilascio degli slot globali e gli aggiornamenti di of_refcount.
p_sem	Terminazione del processo figlio	Semaforo	Sincronizza _exit() e waitpid(): il padre può sospendersi finché il figlio termina, senza busy waiting.

3. Gestione dei file aperti: file descriptor table e openfile

3.1 Requisito della traccia e scelta progettuale

La traccia specifica che le system call sui file non riguardano soltanto l'accesso al file system, ma soprattutto la gestione dei file descriptor e dello stato associato a ciascun processo. In particolare, occorre rappresentare correttamente informazioni come la directory corrente e l'offset di apertura, distinguendo ciò che appartiene al singolo processo da ciò che può essere condiviso tra più descriptor.

Per soddisfare questo requisito, il progetto adotta una struttura a tre livelli:

file descriptor → p_filetable[fd] → struct openfile → struct vnode → VFS

Il *file descriptor* è un identificatore locale del processo; la *openfile* rappresenta una specifica apertura del file e ne conserva lo stato; il *vnode* è invece l'oggetto attraverso cui il VFS rappresenta il file o il device sottostante. Questa separazione permette di gestire correttamente sia aperture indipendenti sia aperture condivise tramite dup2() e fork().

3.2 File descriptor table per processo

Ogni processo dispone di una propria *file descriptor table*, memorizzata direttamente nella struct proc. Il file descriptor non contiene quindi il file stesso, ma è semplicemente l'indice utilizzato per accedere alla relativa entry della tabella.

In kern/include/proc.h:

```
struct openfile *p_filetable[OPEN_MAX];
struct lock *p_filetable_lock;
```

L'array p_filetable contiene puntatori a strutture openfile, mentre p_filetable_lock protegge l'accesso concorrente e le modifiche alla tabella.

In kern/proc/proc.c:

In proc_create() la tabella viene inizializzata con tutti gli slot liberi e viene creato il relativo lock.

```

/*
 * Start with an empty file descriptor table.
 */
for (fd = 0; fd < OPEN_MAX; fd++) {
    proc->p_filetable[fd] = NULL;
}

/*
 * Protect the per-process file descriptor table.
 */
proc->p_filetable_lock =
    lock_create("p_filetable");

```

3.3 struct openfile

Le entry della file descriptor table non puntano direttamente ai vnode, ma a una struttura intermedia denominata openfile, che rappresenta una specifica apertura del file. Questa scelta è necessaria perché più descriptor possono riferirsi alla stessa apertura. In particolare, dup2() e fork() devono poter condividere lo stesso offset, mentre due chiamate open() indipendenti sullo stesso pathname devono mantenere offset separati.

In kern/include/openfile.h:

```

/*
 * System-wide open file description.
 * Multiple file descriptors may refer to the same openfile.
 */
struct openfile {
    struct vnode *of_vnode;           /* Open VFS object */
    off_t of_offset;                  /* Current shared file offset */
    int of_flags;                     /* Opening flags */
    unsigned int of_refcount;          /* Number of active references */
    struct lock *of_lock;              /* Protects open-file state */
};

```

Le strutture openfile non sono allocate direttamente nella tabella del singolo processo, ma vengono mantenute in una tabella globale del kernel, che rappresenta l'insieme delle aperture attive nell'intero sistema.

In kern/syscall/file_syscalls.c:

```

/*
 * Table of the open files currently present in the system.
 * A single openfile may be referenced by more than one descriptor.
 */
static struct openfile system_file_table[SYSTEM_OPEN_MAX];

/*
 * Protects allocation and reference count updates in the system table.
 */
static struct lock *system_file_table_lock;

```

Nel PROGETTO: $\text{SYSTEM_OPEN_MAX} = 10 * \text{OPEN_MAX}$

$\text{OPEN_MAX} = 128 \rightarrow$ quindi la tabella globale può contenere fino a 1280 aperture.

Questa rappresentazione consente di rappresentare correttamente due condizioni di errore differenti:

EMFILE → esauriti i descriptor del singolo processo

ENFILE → esaurite le aperture disponibili a livello di sistema

La creazione di una nuova apertura è gestita da `openfile_open()` (`kern/syscall/file_syscalls.c`):

```
err = vfs_open(kpath, flags, mode, &vn);
```

Per prima cosa viene aperto il vnode tramite VFS.

Poi viene creato il lock associato alla nuova apertura:

```
newlock = lock_create("open file");
```

Dopo di che viene cercato uno slot libero nella tabella globale e avviene l'inizializzazione di una nuova `openfile`:

```
for (i = 0; i < SYSTEM_OPEN_MAX; i++)
{
    if (system_file_table[i].of_vnode == NULL)
    {
        slot = i;
        break;
    }
}

of->of_vnode = vn;
of->of_offset = 0;
of->of_flags = flags;
of->of_refcount = 1;
of->of_lock = newlock;
```

Il reference count parte da 1 perché al momento della creazione esiste già il primo riferimento alla nuova apertura.

REFERENCE COUNTING

Poiché una stessa `openfile` può essere condivisa da più descriptor o da più processi, il suo ciclo di vita è gestito tramite **reference counting**. Quando nasce un nuovo riferimento viene chiamata `openfile_incref()`, mentre quando un riferimento viene eliminato viene utilizzata `openfile_release()`.

In `openfile_release()` (`kern/syscall/file_syscalls.c`):

```
of->of_refcount--;

if (of->of_refcount > 0)
{
    lock_release(system_file_table_lock);
    return;
}
```

La struttura viene quindi mantenuta valida finché esiste almeno un riferimento attivo.

Solo quando il contatore raggiunge zero vengono liberate definitivamente le risorse:

```
of->of_vnode = NULL;
of->of_offset = 0;
of->of_flags = 0;
of->of_refcount = 0;
of->of_lock = NULL;
```

Il reference counting evita quindi che un file venga chiuso mentre è ancora utilizzato da un altro descriptor o processo.

3.4 stdin, stdout, stderr

La traccia richiede che ogni programma inizi con i descriptor 0, 1 e 2, corrispondenti rispettivamente a stdin, stdout e stderr, collegati alla console con:. Deve inoltre essere possibile modificarli successivamente tramite dup2(). Nel progetto questa inizializzazione viene effettuata durante la creazione di un processo destinato all'esecuzione di un programma.

In kern/proc/proc.c, nella funzione proc_create_runprogram():

```
/*  
 * stdin  
 */  
err = openfile_open(  
    "con:",  
    O_RDONLY,  
    0,  
    &newproc->p_filetable[0]);  
  
/*  
 * stdout  
 */  
err = openfile_open(  
    "con:",  
    O_WRONLY,  
    0,  
    &newproc->p_filetable[1]);  
  
/*  
 * stderr  
 */  
err = openfile_open(  
    "con:",  
    O_WRONLY,  
    0,  
    &newproc->p_filetable[2]);
```

Il codice gestisce inoltre gli errori in modo progressivo: se una delle aperture fallisce, vengono rilasciate le risorse eventualmente già allocate prima di distruggere il processo.

4. Implementazione delle system call sui file

Tutte le system call implementate vengono gestite dal dispatcher syscall() in kern/arch/mips/syscall/syscall.c, che identifica la chiamata attraverso il trapframe, recupera gli argomenti e invoca la corrispondente funzione sys_*. Al termine traduce risultato ed eventuale errore secondo la convenzione ABI MIPS e aggiorna il program counter prima del ritorno in user mode.

4.1 open

SYS_OPEN()

La system call open() è implementata in kern/syscall/file_syscalls.c tramite sys_open(). La funzione riceve il pathname come puntatore user space e, come prima operazione, lo copia in un buffer kernel di dimensione PATH_MAX tramite copyinstr(), propagando immediatamente eventuali errori di accesso alla memoria utente.

```
/*  
 * The pathname comes from user space and must be copied safely.  
 */  
err = copyinstr(filename,  
    kernel_path,  
    sizeof(kernel_path),  
    &actual);  
if (err)  
{  
    return err;  
}
```

Successivamente viene estratta e validata la modalità di accesso specificata nei flag, accettando O_RDONLY, O_WRONLY e O_RDWR; una modalità non valida provoca il ritorno di EINVAL.

```

/*
 * Check the access mode.
 */
access_mode = flags & 0_ACCMODE;

switch (access_mode)
{
case 0_RDONLY:
case 0_WRONLY:
case 0_RDWR:
    break;
default:
    return EINVAL;
}

```

Prima di aprire il file, `sys_open()` acquisisce `p_filetable_lock` e ricerca il primo descriptor libero nella file descriptor table del processo. La scansione parte dall'indice 3, poiché gli slot 0, 1 e 2 sono riservati rispettivamente a `stdin`, `stdout` e `stderr`. Se non è disponibile alcuno slot, la system call restituisce `EMFILE` senza procedere all'apertura del file. Questa verifica viene effettuata prima di `openfile_open()` per evitare di eseguire operazioni sul file system quando il processo ha già raggiunto il proprio limite di descriptor.

Poi richiama `openfile_open()`, che apre il file attraverso il VFS e restituisce la relativa `openfile`.

```

lock_acquire(curproc->p_filetable_lock);

fd = -1;

for (i = 3; i < OPEN_MAX; i++)
{
    if (curproc->p_filetable[i] == NULL)
    {
        fd = i;
        break;
    }
}

/*
 * Open the vnode and create its system-wide openfile.
 */
err = openfile_open(kernel_path, flags, mode, &of);
if (err)
{
    lock_release(curproc->p_filetable_lock);
    return err;
}

```

(ogni nuova apertura parte con offset 0 e reference count 1)

Se non è disponibile alcun descriptor, il lock viene rilasciato, la `openfile` appena creata viene liberata tramite `openfile_release()` e viene restituito `EMFILE`. In caso contrario, il puntatore alla `openfile` viene inserito nello slot individuato e il relativo indice viene restituito al processo come nuovo file descriptor.

4.2 read e write

Le system call `write()` e `read()` sono implementate in `kern/syscall/file_syscalls.c` rispettivamente tramite `sys_write()` e `sys_read()`. Entrambe seguono la stessa sequenza generale: validano descriptor, buffer e dimensione; recuperano dalla `p_filetable` la `openfile` associata al descriptor; acquisiscono una reference temporanea per garantirne la validità durante l'I/O; verificano la modalità di apertura; proteggono l'offset condiviso tramite `of_lock`; costruiscono le strutture `iovec` e `uio` richieste dal VFS; eseguono infine `VOP_WRITE()` o `VOP_READ()` e aggiornano l'offset e il numero di byte effettivamente trasferiti.

SYS_WRITE()

Verifica innanzitutto la validità del file descriptor e del buffer utente. Un descriptor fuori intervallo produce EBADF, mentre un buffer nullo con dimensione positiva produce EFAULT. Una richiesta di zero byte termina invece immediatamente con valore di ritorno pari a zero.

```
if (fd < 0 || fd >= OPEN_MAX)
{
    return EBADF;
}

if (buf == NULL && size > 0)
{
    return EFAULT;
}

if (size == 0)
{
    *retval = 0;
    return 0;
}
```

La `openfile` viene recuperata dalla file descriptor table mantenendo `p_filetable_lock`. Prima di rilasciare il lock viene incrementato temporaneamente il reference count tramite `openfile_incref()`: in questo modo un eventuale `close()` concorrente può rimuovere il descriptor dalla tabella, ma non può distruggere la `openfile` mentre l'operazione di I/O è ancora in corso.

```
/*
 * Get the openfile while holding the process table lock.
 * The temporary reference keeps it alive if another thread
 * closes the descriptor while the write is running.
 */
lock_acquire(curproc->p_filetable_lock);

of = curproc->p_filetable[fd];

if (of == NULL)
{
    lock_release(curproc->p_filetable_lock);
    return EBADF;
}

openfile_incref(of);

lock_release(curproc->p_filetable_lock);
```

Viene poi verificato che l'apertura consenta la scrittura: sono accettate soltanto `O_WRONLY` e `O_RDWR`. In caso contrario viene rilasciata la reference temporanea e restituito `EBADF`.

```
access_mode = of->of_flags & O_ACCMODE;

if (access_mode != O_WRONLY && access_mode != O_RDWR)
{
    openfile_release(of);
    return EBADF;
}
```

`of_lock` protegge l'offset della `openfile`, che può essere condiviso da più descriptor o processi. Se il file è stato aperto con `O_APPEND`, prima di ogni scrittura viene letta tramite `VOP_STAT()` la dimensione corrente del file e `of_offset` viene impostato alla sua fine, così da garantire che la scrittura avvenga sempre in coda anche se il file è stato modificato nel frattempo.

```

/*
 * Offset and other openfile state may be shared, so the whole
 * operation is performed while holding the openfile lock.
 */
lock_acquire(of->of_lock);

/*
 * With O_APPEND every write starts from the current end of file.
 */
if (of->of_flags & O_APPEND)
{
    struct stat statbuf;

    result = VOP_STAT(of->of_vnode, &statbuf);
    if (result)
    {
        lock_release(of->of_lock);
        openfile_release(of);
        return result;
    }

    of->of_offset = statbuf.st_size;
}

```

Il buffer utente viene descritto tramite una iovec, mentre la uio specifica offset, quantità residua, direzione UIO_WRITE e address space del processo corrente. Il trasferimento viene quindi delegato al VFS mediante VOP_WRITE().

```

iov.iov_ubase = buf;
iov.iov_len = size;

ku.uio_iov = &iov;
ku.uio_iovcnt = 1;
ku.uio_offset = of->of_offset;
ku.uio_resid = size;
ku.uio_segflg = UIO_USERSPACE;
ku.uio_rw = UIO_WRITE;
ku.uio_space = curproc->p_addrspace;

result = VOP_WRITE(of->of_vnode, &ku);

```

Se la scrittura termina correttamente, l'offset della openfile viene aggiornato con il valore finale della uio, mentre il numero di byte effettivamente scritti è calcolato come size - uio_resid. Infine vengono rilasciati of_lock e la reference temporanea.

```

if (result == 0)
{
    /*
     * VOP_WRITE updates both the offset and the residual count.
     */
    of->of_offset = ku.uio_offset;
    *retval = (int32_t)(size - ku.uio_resid);
}

```

SYS_READ()

sys_read() segue la stessa struttura di sys_write(): valida fd, buffer e dimensione, recupera la openfile sotto p_filetable_lock e ne acquisisce una reference temporanea. La modalità deve essere O_RDONLY oppure O_RDWR; successivamente of_lock protegge l'offset condiviso durante l'intera operazione. La uio viene configurata con UIO_USERSPACE e UIO_READ, utilizzando come offset iniziale of_offset, e il trasferimento viene effettuato tramite VOP_READ(). In caso di successo, l'offset viene aggiornato con uio_offset e il numero di byte letti è calcolato come size - uio_resid.

```

iov.iov_ubase = buf;
iov.iov_len = size;

u.uio_iov = &iov;
u.uio_iovcnt = 1;
u.uio_offset = of->of_offset;
u.uio_resid = size;
u.uio_segflg = UIO_USERSPACE;
u.uio_rw = UIO_READ;
u.uio_space = curproc->p_addrspace;

result = VOP_READ(of->of_vnode, &u);

```

4.3 close

La system call `close()` è implementata in `kern/syscall/file_syscalls.c` tramite `sys_close()`. La funzione inizialmente verifica che il file descriptor sia valido e, sotto protezione di `p_filetable_lock`, recupera la openfile associata; se lo slot è vuoto restituisce `EBADF`. In caso contrario, rimuove subito il descriptor dalla file descriptor table impostando `p_filetable[fd] = NULL`, rendendo così il descrittore nuovamente disponibile al processo. Solo dopo rilascia `p_filetable_lock` e invoca `openfile_release()`, che decrementa il reference count della openfile e ne libera definitivamente `vnode` e `lock` soltanto quando non esistono più riferimenti. Questa separazione è importante perché `close()` elimina il riferimento del singolo processo, ma non deve invalidare un'apertura ancora condivisa tramite `dup2()` o `fork()`.

```

lock_acquire(curproc->p_filetable_lock);

of = curproc->p_filetable[fd];

if (of == NULL)
{
    lock_release(curproc->p_filetable_lock);
    return EBADF;
}

/*
 * The descriptor is no longer available to this process.
 */
curproc->p_filetable[fd] = NULL;

lock_release(curproc->p_filetable_lock);

/*
 * This only removes one reference. The vnode is actually closed
 * when the last reference to the openfile disappears.
 */
openfile_release(of);

```

4.4 lseek

La system call `lseek()` è implementata in `kern/syscall/file_syscalls.c` tramite `sys_lseek()`. Dopo aver validato il file descriptor, la funzione recupera la relativa openfile acquisendone una reference temporanea e verifica con `VOP_ISSEEKABLE()` che il `vnode` supporti il posizionamento; in caso contrario restituisce `ESPIPE`. Il nuovo offset viene calcolato sotto `of_lock`, poiché `of_offset` può essere condiviso tra più descriptor: con `SEEK_SET` coincide con `pos`, con `SEEK_CUR` viene sommato all'offset corrente, mentre con `SEEK_END` viene sommato alla dimensione corrente del file ottenuta tramite `VOP_STAT()`. Un offset finale negativo viene rifiutato con `EINVAL`; altrimenti `of_offset` viene aggiornato e restituito al chiamante.

```

lock_acquire(of->of_lock);

switch (whence)
{
case SEEK_SET:
    newpos = pos;
    break;

case SEEK_CUR:
    newpos = of->of_offset + pos;
    break;

case SEEK_END:
    result = VOP_STAT(of->of_vnode, &st);
    if (result)
    {
        lock_release(of->of_lock);
        openfile_release(of);
        return result;
    }

    newpos = st.st_size + pos;
    break;

default:
    lock_release(of->of_lock);
    openfile_release(of);
    return EINVAL;
}

```

La particolarità di `lseek()` riguarda il parametro `off_t pos`, che in OS/161 è a 64 bit, mentre ciascun registro MIPS contiene 32 bit. La firma è:

`lseek(int fd, off_t pos, int whence)`

Secondo l'ABI MIPS, un argomento a 64 bit deve iniziare su una coppia di registri allineata. Di conseguenza `fd` occupa `a0`, `a1` viene lasciato libero per l'allineamento e i 64 bit di `pos` occupano `a2` e `a3`. Poiché i quattro registri degli argomenti sono ormai esauriti, `whence` viene recuperato dallo stack utente, all'indirizzo `sp + 16`, tramite `copyin()`. Il dispatcher ricostruisce quindi `pos` a 64 bit prima di chiamare `sys_lseek()`. Analogamente, il nuovo offset restituito da `lseek()` è a 64 bit e deve essere diviso in due metà da 32 bit nei registri di ritorno `v0` e `v1`. In questo modo tutta la gestione della convenzione ABI rimane nel dispatcher, mentre `sys_lseek()` lavora direttamente con un normale valore `off_t`.

```

pos = ((off_t)(uint32_t)tf->tf_a2 << 32) |
      (off_t)(uint32_t)tf->tf_a3;

/*
 * No argument register remains for whence.
 * Additional syscall arguments are stored on the
 * user stack after the four register argument slots.
 */
err = copyin(
    (const_userptr_t)(tf->tf_sp + 16),
    &whence,
    sizeof(whence));

```

4.5 dup2

`int sys_dup2(int oldfd, int newfd, int32_t *retval);`

`dup2()` deve fare in modo che `newfd` diventi un secondo file descriptor riferito alla stessa openfile di `oldfd`, sostituendo l'eventuale associazione precedente di `newfd`.

La system call `dup2()` è implementata in `kern/syscall/file_syscalls.c` tramite `sys_dup2()`. La funzione verifica che `oldfd` e `newfd` siano validi e che `oldfd` identifichi effettivamente una `openfile`; il caso `oldfd == newfd` viene trattato come successo senza modifiche. Negli altri casi, sotto `p_filetable_lock`, salva l'eventuale `openfile` già associata a `newfd`, quindi fa puntare `newfd` alla stessa `openfile` di `oldfd` e ne incrementa il reference count. Solo dopo aver rilasciato il lock viene rilasciato l'eventuale vecchio riferimento di `newfd`. In questo modo l'aggiornamento della file descriptor table è atomico e i due descriptor condividono realmente la stessa apertura, quindi anche `vnode`, `flag` e soprattutto `of_offset`: è proprio questa semantica che permette di usare `dup2()` per le redirezioni di `stdin`, `stdout` e `stderr`.

```
/*
 * dup2(fd, fd) is valid and does not change the table.
 */
if (oldfd == newfd)
{
    lock_release(curproc->p_filetable_lock);

    *retval = newfd;
    return 0;
}

/*
 * Keep the old newfd mapping so its reference can be dropped
 * after the table has been updated.
 */
of_new = curproc->p_filetable[newfd];

curproc->p_filetable[newfd] = of_old;

/*
 * oldfd and newfd now refer to the same openfile.
 */
openfile_incref(of_old);
```

4.6 `chdir` e `__getcwd`

`SYS_CHDIR()`

Deve cambiare la current working directory del processo chiamante.

La system call `chdir()` è implementata in `kern/syscall/file_syscalls.c` tramite `sys_chdir()`. Poiché `pathname` proviene dallo user space, viene prima validato e copiato in un buffer kernel tramite `copyinstr()`. Viene inoltre rifiutato un `pathname` vuoto con `EINVAL`. Una volta ottenuta una stringa valida in kernel space, la modifica della directory corrente viene delegata a `vfs_chdir()`, lasciando quindi al VFS la gestione effettiva del `pathname` e della `cwd` associata al processo.

```
err = vfs_chdir(kernel_path);
```

`SYS___GETCWD()`

Deve restituire nello user buffer il `pathname` della current working directory del processo.

La system call `__getcwd()` è implementata tramite `sys___getcwd()`. Dopo aver verificato che il buffer utente non sia `NULL`, costruisce una `iovec` e una `uio` configurata con `UIO_USERSPACE` e `UIO_READ`, indicando direttamente il buffer e l'address space del processo corrente. La `uio` viene quindi passata a `vfs_getcwd()`, che scrive il `pathname` della directory corrente direttamente nello spazio utente. Al termine, il numero di byte effettivamente trasferiti viene calcolato come `buflen - uio_resid` e restituito al chiamante.

Questa soluzione evita l'uso di un buffer kernel intermedio e sfrutta direttamente l'interfaccia uio del VFS.

```
iov.iov_ubase = buf;
iov.iov_len = buflen;

u.uio_iov = &iov;
u.uio_iovcnt = 1;
u.uio_offset = 0;
u.uio_resid = buflen;
u.uio_segflg = UIO_USERSPACE;
u.uio_rw = UIO_READ;
u.uio_space = curproc->p_addrspace;

err = vfs_getcwd(&u);
```

5. Modello dei processi e gestione dei PID

5.1 Estensione di struct proc

Per supportare getpid(), fork(), waitpid() e la gestione corretta della terminazione dei processi, la struttura proc è stata estesa con nuovi campi dedicati all'identità del processo, alla relazione padre-figlio, allo stato di terminazione e alla sincronizzazione. Sono stati inoltre aggiunti i campi relativi alla file descriptor table, già utilizzati nella gestione dei file. In questo modo struct proc diventa la struttura centrale che raccoglie tutto lo stato appartenente a un processo: address space, current working directory, descriptor, PID e informazioni necessarie per attesa e terminazione.

In kern/include/proc.h:

```
#if OPT_PDSC2
    pid_t p_parent_pid;

    bool p_exited;
    bool p_orphan;

    struct openfile *p_filetable[OPEN_MAX];
    struct lock *p_filetable_lock;
#endif
```

Campo	Uso principale
p_pid	Identifica univocamente il processo ed è il valore restituito da getpid().
p_parent_pid	Memorizza il PID del padre e permette a waitpid() di verificare la relazione padre-figlio.
p_status	Conserva lo stato di terminazione del processo, poi restituito al padre tramite waitpid().
p_sem	Sincronizza il padre in waitpid() con la terminazione del figlio, evitando busy waiting.
p_exited	Indica se il processo è già terminato; viene usato anche per la gestione di WNOHANG.
p_orphan	Segnala che il processo non ha più un padre in grado di eseguire waitpid(), permettendone la corretta reclamazione.
p_filetable	Contiene la file descriptor table privata del processo.
p_filetable_lock	Protegge gli accessi e le modifiche concorrenti alla file descriptor table.

5.2 processTable globale e assegnazione dei PID

Per associare ogni PID alla relativa struct proc, in kern/proc/proc.c è stata introdotta una process table globale con al massimo MAX_PROC = 100 processi. L'array ha dimensione MAX_PROC + 1 perché l'indice 0 non viene utilizzato come PID valido: il PID coincide quindi direttamente con l'indice della tabella. L'accesso è protetto da processTable.lk, uno spinlock adatto perché le operazioni effettuate sulla tabella (ricerca, inserimento e rimozione di un puntatore) sono molto brevi e devono essere atomiche. Alla creazione di un processo, proc_init_waitpid() cerca uno slot libero sotto spinlock, registra immediatamente la proc e assegna l'indice a p_pid; se la tabella è piena restituisce ENPROC. Dopo aver rilasciato lo spinlock viene creato p_sem, usato successivamente per sincronizzare waitpid() con la terminazione del processo; se questa allocazione fallisce, l'assegnazione del PID viene annullata rendendo nuovamente libero lo slot. proc_search_pid() realizza invece l'operazione inversa, recuperando dalla tabella la struct proc corrispondente a un determinato PID. Il PID resta occupato fino alla distruzione definitiva della proc, evitando che venga riutilizzato mentre lo stato di un processo terminato deve ancora essere raccolto dal padre.

```
static struct _processTable {
    int active;
    struct proc *proc[MAX_PROC + 1];
    int last_i;
    struct spinlock lk;
} processTable;
```

proc_init_waitpid():

```
spinlock_acquire(&processTable.lk);

for (i = 1; i <= MAX_PROC; i++) {
    if (processTable.proc[i] == NULL) {
        processTable.proc[i] = proc;
        processTable.last_i = i;

        proc->p_pid = i;
        pid_found = 1;
        break;
    }
}

spinlock_release(&processTable.lk);

proc->p_sem = sem_create(name, 0);
if (proc->p_sem == NULL) {
    spinlock_acquire(&processTable.lk);
    if (processTable.proc[proc->p_pid] == proc) {
        processTable.proc[proc->p_pid] = NULL;
    }
    spinlock_release(&processTable.lk);
    proc->p_pid = 0;
    return ENOMEM;
}
```

proc_search_pid():

```
spinlock_acquire(&processTable.lk);
p = processTable.proc[pid];
spinlock_release(&processTable.lk);
```

SYS_GETPID()

PROTOTIPO: pid_t sys_getpid(void);

La system call getpid() è implementata in kern/syscall/proc_syscalls.c tramite sys_getpid(). Poiché il PID viene assegnato alla creazione del processo e memorizzato direttamente nel campo

p_pid della struct proc, la funzione non deve effettuare alcuna ricerca nella process table: verifica semplicemente che curproc esista e restituisce curproc->p_pid.

6. fork

Deve creare un nuovo processo figlio che inizialmente sia una copia del padre; dopo la chiamata il padre riceve il PID del figlio, mentre il figlio riceve 0.

PROTOTIPO: `int sys_fork(struct trapframe *ctf, pid_t *retval);`

La fork() è implementata in kern/syscall/proc_syscalls.c. L'implementazione procede in questo ordine: crea innanzitutto la struct proc del figlio tramite proc_create_fork(), che assegna un nuovo PID, memorizza il PID del padre ed eredita la current working directory.

```
create_err = 0;
child = proc_create_fork(curproc->p_name, &create_err);

if (child == NULL)
{
    return create_err != 0 ? create_err : ENOMEM;
}
```

In proc_create_fork():

```
if (curproc->p_cwd != NULL) {
    VOP_INCREC(curproc->p_cwd);
    newproc->p_cwd = curproc->p_cwd;
}
```

Successivamente as_copy() crea una copia indipendente dell'address space del padre.

```
result = as_copy(curproc->p_addrspace,
                &child_as);

if (result)
{
    proc_destroy(child);
    return result;
}

child->p_addrspace = child_as;
```

La file descriptor table viene poi copiata con proc_filetable_copy(): padre e figlio possiedono tabelle distinte, ma i descriptor ereditati puntano alle stesse openfile, incrementandone il reference count e condividendo quindi anche l'offset.

```
proc_filetable_copy(curproc, child);
```

In proc_filetable_copy():

```

if (src->p_filetable[fd] != NULL) {
    dst->p_filetable[fd] =
        src->p_filetable[fd];
    /*
     * The child owns another reference to
     * the same open file.
     */
    openfile_incref(
        dst->p_filetable[fd]);
}

```

Infine viene copiato il trapframe del padre e `thread_fork()` crea il thread che eseguirà il processo figlio. In caso di successo, al padre viene restituito `child->p_pid`.

```

/*
 * Copy the trapframe.
 */
child_tf = kmalloc(sizeof(struct trapframe));

if (child_tf == NULL)
{
    proc_destroy(child);
    return ENOMEM;
}

memcpy(child_tf,
        ctf,
        sizeof(struct trapframe));

/*
 * Create the child thread.
 */
result = thread_fork(curproc->p_name,
                    child,
                    call_enter_forked_process,
                    child_tf,
                    0);

```

```

/*
 * Parent receives the child PID.
 */
*retval = child->p_pid;

return 0;

```

Padre e figlio riprendono dallo stesso punto con valori di ritorno diversi:

- PADRE → pid del figlio
- FIGLIO → 0

Infatti in `enter_forked_process()`:

```

/*
 * Copy the saved parent CPU state onto the child's
 * kernel stack and release the temporary heap copy.
 */
child_tf = *tf;
kfree(tf);

/*
 * fork() returns 0 in the child.
 */
child_tf.tf_v0 = 0;

```

```

/*
 * a3 = 0 indicates a successful syscall.
 */
child_tf.tf_a3 = 0;

```

7. execv

Deve sostituire il programma eseguito dal processo corrente con un nuovo eseguibile, mantenendo però lo stesso processo. Quindi non crea un nuovo PID: restano invariati PID, parentela, cwd e file descriptor, mentre vengono sostituiti address space, codice, dati e stack.

PROTOTIPO: `int sys_execv(userptr_t progname, userptr_t argv);`

`sys_execv()`, implementata in `kern/syscall/proc_syscalls.c`, sostituisce l'immagine del processo corrente senza crearne uno nuovo. La funzione copia innanzitutto dal user space il pathname dell'eseguibile e l'intero vettore `argv`, controllando il limite `ARG_MAX`.

```
/*
 * Copy the executable pathname into kernel space.
 */
result = copyinstr(progname,
                   kernel_prog,
                   sizeof(kernel_prog),
                   &actual);
```

```
result = copyin(
    (const_userptr_t)((vaddr_t)argv +
                      argc * sizeof(userptr_t)),
    &user_arg,
    sizeof(user_arg));
```

```
/*
 * Prevent an invalid argv from growing indefinitely.
 */
if ((size_t)argc *
    sizeof(userptr_t) >=
    ARG_MAX)
{
    return E2BIG;
}
```

```
result = copyin(
    (const_userptr_t)((vaddr_t)argv +
                      i * sizeof(userptr_t)),
    &user_arg,
    sizeof(user_arg));
```

Successivamente apre il nuovo eseguibile, crea un nuovo address space e lo installa temporaneamente nel processo per consentire a `load_elf()` di caricare il programma.

```
/*
 * Create a new address space.
 */
newas = as_create();
oldas = proc_setas(newas);
as_activate();

result = load_elf(v, &entrypoint);
```

Non viene distrutto subito il vecchio address space perché se `load_elf()` fallisce si deve poter tornare al programma precedente.

Dopo aver creato il nuovo stack, ricostruisce su di esso le stringhe degli argomenti e il vettore argv, rispettandone l'allineamento.

```
/*
 * Create the new stack.
 */
result = as_define_stack(newas,
    &stackptr);
```

```
/*
 * Keep strings aligned to 8 bytes.
 */
stackptr -= ROUNDUP(len, 8);

result = copyoutstr(
    kargv[i],
    (userptr_t)stackptr,
    len,
    NULL);
```

```
uargv[i] = stackptr;
```

```
uargv[argc] = 0;

/*
 * Reserve space for argv itself.
 */
stackptr -= (argc + 1) * sizeof(vaddr_t);

/*
 * Align argv.
 */
stackptr &= ~(vaddr_t)7;

argv_user = stackptr;

/*
 * Copy argv[] onto the new user stack.
 */
result = copyout(
    uargv,
    (userptr_t)argv_user,
    (argc + 1) * sizeof(vaddr_t));
```

Solo quando la nuova immagine è stata costruita correttamente viene distrutto il vecchio address space; infine `enter_new_process()` trasferisce l'esecuzione al nuovo programma in user mode. In caso di errore durante la costruzione, il vecchio address space viene ripristinato, evitando di lasciare il processo in uno stato parzialmente modificato.

```
/*
 * The new program image is complete.
 */
if (oldas != NULL)
{
    as_destroy(oldas);
}

enter_new_process(
    argc,
    (userptr_t)argv_user,
    NULL,
    stackptr,
    entrypoint);
```

Se la `execv()` ha successo non ritorna alla vecchia istruzione chiamante, ma `enter_new_process()` entra direttamente nel nuovo programma.

7.1 Differenza rispetto a `runprogram`

`runprogram()` avvia un programma da kernel su un processo senza address space; `execv()` sostituisce invece il programma di un processo già esistente. Per questo `execv()` deve copiare

progname e argv, mantenere PID, cwd e file descriptor, e ripristinare il vecchio address space se qualcosa fallisce.

8. waitpid, _exit e ciclo di vita

8.1 _exit() e stato zombie

SYS__EXIT()

Deve terminare il processo corrente, salvandone lo stato di uscita affinché possa essere successivamente recuperato dal padre tramite waitpid().

PROTOTIPO: void sys__exit(int status); (implementata in kern/syscall/proc_syscalls.c)

sys__exit() termina il processo corrente salvando innanzitutto lo stato in p_status tramite _MKWAIT_EXIT(status). L'address space viene quindi scollegato dal processo e distrutto, poiché non sarà più eseguito codice user. Il thread viene rimosso dalla proc prima di completare la terminazione tramite proc_finish_exit(), evitando che la struttura possa essere distrutta mentre contiene ancora thread attivi. Infine thread_exit() termina definitivamente il thread.

```
p->p_status = _MKWAIT_EXIT(status);

/*
 * The user address space is no longer needed.
 */
as = proc_setas(NULL);
as_deactivate();

if (as != NULL) {
    as_destroy(as);
}

proc_remthread(curthread); proc_finish_exit(p);

thread_exit();
```

PROC_FINISH_EXIT()

Deve completare la terminazione della proc dopo che l'ultimo thread è già stato rimosso, rendendo il processo osservabile dal padre tramite waitpid().

(in kern/proc/proc.c)

proc_finish_exit() viene chiamata dopo proc_remthread(), quindi verifica che p_numthreads == 0 e imposta p_exited = true. Se il processo ha ancora un padre, la struct proc non viene distrutta: rimane nella process table e il padre viene notificato tramite V(p_sem). Il processo si trova così nello stato equivalente a uno zombie: non possiede più address space né thread eseguibili, ma conserva PID e p_status finché il padre non li raccoglie con waitpid().

```
/*
 * The process has completely terminated.
 */
proc->p_exited = true;
V(proc->p_sem);
```

8.2 waitpid() e sincronizzazione

Viene implementata in kern/syscall/proc_syscalls.c dalla funzione sys_waitpid().

sys_waitpid() permette al padre di attendere e raccogliere lo stato di terminazione di un proprio figlio. La funzione verifica le opzioni, ricerca il processo tramite PID e controlla la relazione padre-figlio. Con options = 0 l'attesa è bloccante; con WNOHANG, se il figlio è ancora attivo, la chiamata restituisce subito 0. Quando il figlio è terminato, proc_wait() ne recupera p_status, che viene copiato in user space con copyout(). Solo dopo un copyout() riuscito viene eseguita proc_destroy(), liberando definitivamente il processo e il relativo PID.

```
/*
 * Find the requested process.
 */
child = proc_search_pid(pid);

if (child == NULL) {
    return ESRCH;
}

if (child->p_parent_pid != curproc->p_pid) {
    return ECHILD;
}
```

CASO WNOHANG:

```
if (options == WNOHANG && !proc_has_exited(child)) {

    *retval = 0;

    return 0;
}
```

CASO NORMALE:

```
status = proc_wait(child);

if (statusp != NULL) {
    result = copyout(&status,
                    statusp,
                    sizeof(status));
    /*
     * The exit status has been collected successfully.
     */
    proc_destroy(child);
    *retval = pid;
}
```

PROC_WAIT()

proc_wait() sincronizza il padre con la terminazione del figlio tramite p_sem, inizializzato a 0. La P() sospende il padre se il figlio è ancora attivo; quando il figlio termina, proc_finish_exit() esegue V() e lo risveglia. Dopo aver letto p_status, proc_wait() esegue nuovamente V() per mantenere disponibile la notifica nel caso in cui il successivo copyout() di waitpid() fallisca e l'attesa debba essere ripetuta. In questo modo non viene utilizzato busy waiting.

```
P(proc->p_sem);

return_status = proc->p_status;

/*
 * Restore the semaphore notification.
 *
 * If copyout() later fails, waitpid() can be called again.
 */
V(proc->p_sem);
```

8.3 Gestione degli orfani

Un processo diventa orfano quando il padre termina prima di lui. In `proc_finish_exit()` vengono individuati i figli del processo terminato: per ciascuno viene azzerato `p_parent_pid` e impostato `p_orphan`.

```
child->p_parent_pid = 0;
child->p_orphan = true;
```

Se il figlio era già terminato, la sua `proc` può essere distrutta subito; se è ancora attivo, verrà distrutta quando terminerà. In questo modo non rimangono zombie senza un padre che possa eseguire `waitpid()`.

```
/*
 * If the child had already exited, nobody will ever
 * call waitpid() for it, so reclaim it now.
 */
if (child->p_exited) {
    processTable.proc[i] = NULL;

    to_destroy[destroy_count] = child;
    destroy_count++;
}

/*
 * If this process itself had already become an orphan,
 * no parent will ever collect it.
 */
if (proc->p_orphan) {
    if (proc->p_pid > 0 &&
        proc->p_pid <= MAX_PROC &&
        processTable.proc[proc->p_pid] == proc) {
        processTable.proc[proc->p_pid] = NULL;
    }

    destroy_self = true;
}
```

8.4 Integrazione con il menu kernel

Il menu non dovrebbe ritornare immediatamente dopo aver avviato un programma, ma dovrebbe attenderne la terminazione. `common_prog()` crea quindi la nuova `proc`, avvia il relativo thread e utilizza `proc_wait()` per sospendere il menu fino alla fine del programma. Questo evita anche una race sull'array `args`, che viene utilizzato dal nuovo thread mentre appartiene ancora al menu.

In `kern/main/menu.c`:

```
/* Create a process for the new program to run in. */
proc = proc_create_runprogram(args[0] /* name */);
if (proc == NULL) {
    return ENOMEM;
}

result = thread_fork(args[0] /* thread name */,
    proc /* new process */,
    cmd_progthread /* thread function */,
    args /* thread arg */, nargs /* thread arg */);
if (result) {
    kprintf("thread_fork failed: %s\n", strerror(result));
    proc_destroy(proc);
    return result;
}

proc_wait(proc);
proc_destroy(proc);
```

9. Gestione delle eccezioni fatali

KILL_CURTHREAD()

kill_curthread() gestisce le eccezioni fatali generate da un programma utente. Il codice dell'eccezione MIPS viene tradotto nel corrispondente segnale Unix, ad esempio SIGSEGV, SIGBUS, SIGILL o SIGFPE. Lo stato di terminazione viene salvato in p_status tramite _MKWAIT_SIG(sig), distinguendo così una terminazione per segnale da una normale _exit(). Successivamente viene distrutto l'address space, viene notificato l'eventuale padre in attesa e il thread viene terminato definitivamente con thread_exit().

In kern/arch/mips/locoer/trap.c:

```
switch (code) {  
    case EX_IRQ:  
    case EX_IBE:  
    case EX_DBE:  
    case EX_SYS:  
        /*  
         * These exceptions should never reach kill_curthread().  
         */  
        KASSERT(0);  
        sig = SIGABRT;  
        break;  
  
    case EX_MOD:  
    case EX_TLBL:  
    case EX_TLBS:  
        sig = SIGSEGV;  
        break;
```

```
    case EX_ADEL:  
    case EX_ADES:  
        sig = SIGBUS;  
        break;  
  
    case EX_BP:  
        sig = SIGTRAP;  
        break;  
  
    case EX_RI:  
        sig = SIGILL;  
        break;  
  
    case EX_CPU:  
        sig = SIGSEGV;  
        break;  
  
    case EX_OVF:  
        sig = SIGFPE;  
        break;  
}  
p->p_status = _MKWAIT_SIG(sig);
```

```

as = proc_setas(NULL);
as_deactivate();

if (as != NULL) {
    as_destroy(as);
}

```

```

    V(p->p_sem);
#endif

/*
 * thread_exit() removes the current thread from the process
 * and permanently stops its execution.
 */
thread_exit();

```

10. Error handling e atomicità

10.1 Validazione dei puntatori utente

I puntatori provenienti dallo user space non vengono mai dereferenziati direttamente dal kernel. I pathname vengono acquisiti tramite `copyinstr()`, i dati semplici tramite `copyin()`, mentre `copyout()` e `copyoutstr()` sono utilizzate per trasferire dati dal kernel allo user space. Queste primitive validano gli indirizzi utente e consentono alle system call di restituire `EFAULT` in caso di puntatore non valido, evitando che un errore dell'applicazione provochi un fault in kernel mode.

10.2 Errori restituiti

Chiamata	Classi di errore	Comportamento di recupero
open	EFAULT, EINVAL, EMFILE, ENFILE e errori VFS	Valida puntatore e flag; distingue il limite per-processo da quello globale.
read/write	EBADF, EFAULT e errori VOP	Controlla fd, modalità di apertura e buffer; propaga gli errori di I/O.
lseek	EBADF, ESPIPE, EINVAL	Rifiuta oggetti non seekable, whence non valido e posizione finale negativa.
close/dup2	EBADF	Verifica validità e presenza del file descriptor.
chdir/__getcwd	EFAULT e errori VFS	Delega al VFS la gestione del pathname e della directory corrente.
fork	ENPROC, ENOMEM, errori di <code>as_copy()</code> / <code>thread_fork()</code>	Distrugge il figlio incompleto prima di restituire l'errore.
execv	EFAULT, EINVAL, E2BIG, ENOMEM e errori VFS/ELF	In caso di errore ripristina il vecchio address space.
waitpid	EINVAL, ESRCH, ECHILD, EFAULT	Non distrugge il figlio se il <code>copyout()</code> dello status fallisce.

11. Verifica funzionale

Test	Aspetto verificato	Esito atteso/verificato
badcall	Gestione di fd, puntatori e argomenti non validi	Restituzione degli errori senza crash del kernel
forktest	Creazione ripetuta di processi figli	PID distinti, corretta duplicazione dell'address space e terminazione dei figli
multiexec	Esecuzioni ripetute di <code>execv()</code>	Sostituzione corretta dell'immagine del processo e gestione di <code>argv</code>
bigseek	<code>lseek()</code> con offset a 64 bit	Corretta gestione dell'ABI MIPS e degli offset
redirect	<code>dup2()</code> e redirezione dei descrittori standard	Redirezione mantenuta anche attraverso <code>fork()</code> ed <code>execv()</code>
/bin/sh	Integrazione complessiva delle system call	Esecuzione corretta di programmi come <code>cat</code> , <code>cp</code> , <code>false</code> , <code>pwd</code> e <code>true</code>

12. Conclusioni e difficoltà riscontrate

Le difficoltà principali del progetto hanno riguardato soprattutto la corretta gestione dello stato condiviso e del ciclo di vita delle risorse. Per i file è stato necessario introdurre la struttura `openfile`, separandola dalla file descriptor table del processo, così da permettere a `fork()` e `dup2()` di condividere correttamente offset e stato dell'apertura tramite reference counting. Proprio la gestione dei riferimenti è stata una delle prime criticità emerse durante i test, con errori dovuti a decrementi non corretti del `refcount`.

Un secondo aspetto delicato è stato la concorrenza: invece di utilizzare un unico lock globale sono stati impiegati `spinlock` e lock distinti per process table, file table e singole `openfile`, proteggendo solo lo stato effettivamente condiviso. Anche `lseek()` ha richiesto particolare attenzione per l'ABI MIPS a 32 bit, dovendo gestire un `off_t` a 64 bit tra registri e stack.

La realizzazione di `execv()` ha richiesto di copiare `pathname` e argomenti dallo user space prima di sostituire l'address space e di mantenere temporaneamente quello precedente, così da poter effettuare il rollback in caso di errore. Analogamente, nella gestione di `_exit()` e `waitpid()` è emersa durante `forktest` una race condition: il padre poteva tentare di distruggere il figlio quando questo aveva ancora un thread associato. La soluzione finale fa quindi rimuovere prima il thread dalla proc in `thread_exit()` e solo successivamente completa la terminazione con `proc_finish_exit()`. È stata inoltre curata la sincronizzazione tramite semaforo, mantenendo la possibilità di ripetere `waitpid()` se il `copyout()` dello status fallisce.

Ulteriori problemi hanno riguardato la corretta restituzione di `ENPROC` quando la process table è piena, la gestione degli orfani e del riutilizzo dei PID, e in generale il cleanup dei percorsi di errore, evitando di lasciare allocati PID, vnode, lock, address space o `openfile`. Nel complesso, il progetto ha quindi richiesto non solo l'implementazione delle singole system call, ma soprattutto il coordinamento corretto tra file, processi, memoria e sincronizzazione, mantenendo stabile il kernel anche in presenza di errori provenienti dallo user space.